OFFENSIVE SECURITY TO SECURE ENGINEERING

8-Week Pipeline Execution Guide

Candidate: Aditya

Red Team Thinking → Secure SDLC Automation

Pipeline Philosophy

This is NOT a generic cybersecurity roadmap. It is a structured offensive-to-defensive engineering pipeline designed to build deep exploit intuition and translate that into systematic, automated secure software engineering.

> **Core Principle**
> Understand exploitation deeply enough to engineer secure systems systematically. You cannot defend what you do not understand how to break.

What This Pipeline Produces

- Exploit fluency: manual web + binary + API attack chains
- Vulnerable target: intentionally insecure backend you build yourself
- Automated analysis: Semgrep + CodeQL + Atheris fuzzer pipeline
- Patched architecture: structurally hardened refactor
- CI/CD enforcement: regression gates that fail on high-severity findings
- Professional AppSec report: threat model, RCA, fixes, architecture review

Industry Role Mapping

Role	Skills Overlap
`Application Security Engineer`	SAST, code review, threat modelling, patch design
`Product Security Engineer`	Secure SDLC, CI/CD gates, design review
`Offensive Security Engineer`	Manual exploitation, vulnerability research
`DevSecOps Engineer`	Pipeline automation, toolchain integration
`Security Researcher`	Fuzz harness engineering, taint flow analysis

Pre-Requisite Toolchain Setup

Install and verify all tools before Day 1. Every lab in Weeks 1-8 depends on this foundation.

1. Burp Suite Community Edition

Burp is your primary proxy — the single most important tool for web security work. It sits between your browser and the server, letting you intercept, inspect, modify, and replay every HTTP request.

Install

- Download from: https://portswigger.net/burp/communitydownload
- Run installer, launch Burp, select Temporary Project

Key Configuration

- Go to Proxy > Options. Listener must be 127.0.0.1:8080
- Open Burp's embedded Chromium: Proxy > Intercept > Open Browser
- This browser routes all traffic through Burp automatically

Verify it works

```
Open embedded browser > visit http://example.com
Proxy > HTTP History should show the request
```

Why embedded browser?

Avoids certificate errors for HTTPS interception. No separate CA cert install needed. Stick to it for all labs.

2. Docker Engine

Docker runs isolated containers — you will deploy vulnerable apps (crAPI, your own API) without polluting your host system.

Linux / WSL2 Install

```
sudo apt update && sudo apt install -y docker.io docker-compose
sudo usermod -aG docker $USER && newgrp docker
docker run hello-world   # verify
```

Windows

- Install Docker Desktop from https://www.docker.com/products/docker-desktop
- Enable WSL2 backend in Settings

WSL2 Note

All binary exploitation (Week 1 Stage 2) requires a Linux environment. Use WSL2 Ubuntu 22.04 or a Kali VM. Install the same Docker inside WSL2.

3. VS Code + Extensions

```
code --install-extension ms-python.python
code --install-extension ms-vscode.cpptools
```

```
code --install-extension esbenp.prettier-vscode
code --install-extension redhat.vscode-yaml
```

4. Linux Environment Check

```
uname -a          # should be Linux kernel
python3 --version # need 3.9+
gcc --version     # need for binary labs
gdb --version     # GNU debugger for binary analysis
```

WEEK 1 — Vulnerability Mechanics & Applied Exploitation

Week 1 is pure offensive training. You are learning the attacker's mindset — how vulnerabilities actually work at the HTTP, memory, and API layers. Every concept here feeds directly into the detection and patching work of Weeks 2-8.

Stage 1: Web & Logic Mechanics (Days 1-2)

Platform: PortSwigger Web Security Academy (https://portswigger.net/web-security)

Core Concept: What is an Intercepting Proxy?

Every web request your browser sends is normally invisible to you. Burp inserts itself as a man-in-the-middle on your own machine — your browser sends to Burp (port 8080), Burp forwards to the server, gets the response, hands it back. This lets you read, pause, and modify requests before they reach the server.

Mental Model

Think of Burp as a glass pipe. You can see everything flowing through it, and you can squeeze it to reshape the data before it exits.

Task 1: Configure Intercepting Proxy

Step-by-Step

1. Launch Burp Suite, create Temporary Project, use Burp defaults
2. Navigate to: Proxy > Intercept tab. Click 'Open Browser'
3. In the embedded browser, navigate to https://portswigger.net
4. Switch Intercept to ON. Reload the page — Burp will freeze the request
5. You can now read every header, cookie, body param. Click Forward to release
6. Right-click any intercepted request > Send to Repeater
7. In Repeater: modify a parameter value, click Send, compare responses

Repeater vs Intercept

Intercept: real-time pause/modify. Repeater: replay a saved request as many times as you want with different inputs. Repeater is where most testing happens.

Task 2: SQL Injection (SQLi)

Concept

SQL Injection happens when user input is concatenated directly into a SQL query without sanitization. The database cannot distinguish attacker-controlled SQL from legitimate query logic.

Example of vulnerable code (Python)

```
query = "SELECT * FROM users WHERE username='" + username + "'"
```

If username = ' OR '1'='1, the query becomes:

```
SELECT * FROM users WHERE username='' OR '1'='1'
```

Since '1'='1' is always true, this returns ALL users — authentication bypassed.

Lab Targets on PortSwigger

- SQL injection vulnerability in WHERE clause allowing retrieval of hidden data
- SQL injection vulnerability allowing login bypass
- SQL injection UNION attack — determining number of columns
- SQL injection attack — querying the database type and version

Lab Execution Methodology

8. Open lab in embedded Burp browser
9. Intercept the login or search request
10. Send to Repeater
11. Inject payloads systematically:

```
'                    -- single quote: causes DB error if vulnerable
' OR '1'='1         -- auth bypass
' UNION SELECT NULL--  -- test UNION column count
' UNION SELECT NULL,NULL--
' UNION SELECT table_name,NULL FROM information_schema.tables--
```

12. Observe HTTP response codes and body diffs between requests

> **Key Insight**
> A 500 error on ' injection means the DB is choking on your unescaped quote. A 200 with different content means your logic changed what the query returns. Read the response body carefully.

Task 3: Cross-Site Scripting (XSS)

Concept

XSS is client-side injection — you inject JavaScript into a page that another user's browser executes. The server reflects or stores your payload, and the victim's browser runs it trusting it as legitimate site code.

Types

Type	Mechanism
`Reflected XSS`	Payload in URL/request, echoed immediately in response. Victim must click crafted link.
`Stored XSS`	Payload saved in DB (comments, profile fields). Executes for every user who views it.

DOM-based XSS	JavaScript reads attacker data from URL/storage and writes it to DOM unsafely.

Test Payloads

```
<script>alert(1)</script>
<img src=x onerror=alert(document.cookie)>
"><script>alert(1)</script>
javascript:alert(1)
```

Lab Methodology

13. Find any input field that reflects back to the page (search, comments, username)
14. Inject basic payload: <script>alert(1)</script>
15. If blocked, try attribute injection: " onmouseover="alert(1)
16. In Burp, compare the raw HTML response — find where your input lands
17. If inside a JS string, break out: '; alert(1);//

Task 4: BOLA / IDOR (Broken Object Level Authorization)

Concept

BOLA (also called IDOR — Insecure Direct Object Reference) is the most common API vulnerability. The server exposes object IDs (user IDs, order IDs) in URLs/parameters but fails to verify that the requesting user actually owns that object.

Example

```
GET /api/users/1042/orders    <- you are user 1042
GET /api/users/1041/orders    <- you access another user's orders
```

If the server returns 1041's data to you without an authorization check, that is BOLA.

Lab Methodology

18. Log in as user A. Note your user ID in the URL or response
19. Capture a request to your own resource in Burp Repeater
20. Modify the ID to an adjacent value (decrement or increment by 1)
21. Send the request — if you get another user's data: BOLA confirmed
22. Try common patterns: /api/users/FUZZ, /account?id=FUZZ, /orders/FUZZ

> **Why BOLA is #1 on OWASP API Top 10**
>
> It is not a code bug — it is a missing authorization check. Developers often assume 'only logged-in users get valid IDs' but forget to verify ownership. Easy to miss in code review, trivial to exploit.

Task 5: Server-Side Request Forgery (SSRF)

Concept

SSRF tricks the server into making HTTP requests on your behalf. If the server fetches URLs from user input without validation, you can make it reach internal services (169.254.169.254 metadata API, internal admin panels, databases) that you cannot reach directly from the internet.

Classic Target: AWS Metadata

```
Normal: POST /fetch?url=https://example.com
Attack: POST /fetch?url=http://169.254.169.254/latest/meta-data/
        -> returns AWS instance role credentials
```

Lab Methodology

23. Find any feature that fetches remote content: URL preview, webhook, PDF export, image URL
24. Intercept the request in Burp, find the URL parameter
25. Replace URL with http://localhost/ or http://127.0.0.1/admin
26. Try cloud metadata: http://169.254.169.254/latest/meta-data/
27. Try internal port scanning: http://127.0.0.1:6379/ (Redis), :5432 (Postgres)

Stage 2: Memory Corruption (Days 3-4)

Platforms: OverTheWire Narnia (https://overthewire.org/wargames/narnia), PicoCTF (https://picoctf.org)

Concept: C Memory Layout

Every running C program has its memory organized into regions. Understanding this layout is prerequisite to all binary exploitation.

Region / Register	Description
`Stack`	Local variables, function call frames, return addresses. Grows downward. Fixed size per frame.
`Heap`	Dynamic allocations (malloc/new). Grows upward. Managed by allocator.
`Text`	Read-only executable code (.text section).
`BSS / Data`	Global and static variables, initialized and uninitialized.
`EIP / RIP`	Instruction pointer register — holds address of NEXT instruction to execute.

The key to buffer overflow

Local variables live on the stack. Below them (at higher address) sits the saved return address — where execution resumes when the function returns. If you write past a buffer's boundary, you overwrite that return address.

Task 1: Stack-Based Buffer Overflow

Vulnerable C Code (compile and analyze)

```
#include <string.h>
void vuln(char *input) {
    char buf[64];
    strcpy(buf, input);  // no bounds check!
}
int main(int argc, char *argv[]) {
    vuln(argv[1]);
}
```

Compile with protections disabled (for learning)

```
gcc -fno-stack-protector -z execstack -no-pie -o vuln vuln.c
```

Step-by-Step Exploitation

28. Find buffer size: 64 bytes
29. Find offset to return address: 64 (buffer) + 8 (saved RBP) = 72 bytes on 64-bit
30. Generate a cyclic pattern to confirm offset:
    ```
    python3 -c "print('A'*72 + 'BBBBBBBB')" | ./vuln
    ```
31. In GDB, examine the crash:
    ```
    gdb ./vuln
    run $(python3 -c "print('A'*72 + 'BBBBBBBB')")
    info registers rip   # should show 0x4242424242424242 (BBBB...)
    ```
32. Replace BBBBBBBB with the address you want to jump to

OverTheWire Narnia Labs

- SSH: ssh -p 2226 narnia0@narnia.labs.overthewire.org
- Password for narnia0: narnia0
- Read source: cat /narnia/narnia0.c — understand what it does
- Exploit: craft input to overwrite the target variable or return address
- Progressive difficulty: narnia0 through narnia9

GDB Cheatsheet

break main | run <args> | disas vuln | info registers | x/20wx $rsp | nexti | stepi | continue | quit

Task 2: Control Flow Redirection

Goal

Once you control RIP (return instruction pointer), redirect execution to a win function or injected shellcode.

Method 1: Return to win function

```
# Find address of target function
objdump -d vuln | grep win
# Craft payload
```

```
python3 -c "import struct; print('A'*72 + struct.pack('<Q', 0x401234))" > payload
./vuln $(cat payload)
```

Method 2: Shellcode injection

```
# Only works with execstack + no ASLR disabled:
echo 0 > /proc/sys/kernel/randomize_va_space
# Inject shellcode in buffer, redirect to buffer address
```

Stage 3: Full-System API Exploitation (Days 5-6)

Platform: OWASP crAPI (Completely Ridiculous API) — https://github.com/OWASP/crAPI

Task 1: Deploy crAPI

```
git clone https://github.com/OWASP/crAPI.git
cd crAPI
docker-compose -f deploy/docker/docker-compose.yml up -d
# Wait 2-3 minutes for all services to start
docker-compose ps    # verify all containers are 'Up'
```

Access the UI at: http://localhost:8888

API directly at: http://localhost:8888/identity/api/...

Task 2: Map the Attack Surface

Why mapping comes before exploiting

Black-box testing requires you to understand the full application before probing it. Mapping gives you: every endpoint, every parameter, every data relationship. You cannot find BOLA if you do not know all the object types exposed.

Methodology

33. Register 2 user accounts (attacker and victim)
34. Browse all features with Burp Proxy enabled — every click creates HTTP history
35. In Burp: Proxy > HTTP History. Export or document all unique endpoints
36. Note the pattern of object IDs (are they sequential integers? UUIDs?)
37. Build an endpoint inventory table:

Endpoint	Notes / Attack Surface
`GET /identity/api/v2/user/dashboard`	Returns current user profile — contains user ID
`GET /identity/api/v2/vehicle/vehicles`	Returns vehicles for current user
`GET /community/api/v2/community/posts/{post_id}`	Get post by ID — test BOLA here

`PUT /identity/api/v2/user/change-email`	Change email — test mass assignment
`POST /workshop/api/mechanic/receive_report`	SSRF vector — sends HTTP request

Task 3: BOLA for Data Exfiltration

38. Log in as user A. Note your user ID from /dashboard response
39. Capture GET /identity/api/v2/vehicle/{vehicleId}/location in Burp
40. Send to Repeater. Change vehicleId to a value belonging to user B
41. If you receive user B's location data: BOLA confirmed
42. Try the same pattern on /community/api/v2/community/posts/{post_id}

```
# Example: you are user ID 3, try:
GET /workshop/api/shop/orders/1
GET /workshop/api/shop/orders/2
GET /workshop/api/shop/orders/3   <- yours
```

Task 4: Mass Assignment

Concept

Mass assignment occurs when an API automatically maps all JSON fields in a request body to a server-side object without whitelisting. If the model has a privileged field (isAdmin, creditBalance, role), an attacker can inject it in the request.

Example

```
// Normal registration
POST /api/register
{"name": "Aditya", "email": "a@a.com", "password": "pw"}

// Mass assignment attack — inject extra field
{"name": "Aditya", "email": "a@a.com", "password": "pw",
 "isAdmin": true, "credit": 99999}
```

43. In crAPI, find the vehicle add endpoint or coupon endpoint
44. Capture the normal request. Note the JSON body
45. Add extra fields: "credit": 9999 or "available_credit": 9999
46. Observe if the server accepts and applies the injected field

Stage 4: Code-to-Exploit Synthesis (Day 7)

Today you connect what you exploited to the source code that enabled it. This is the transition from red team to security engineer.

Task 1: Trace Exploits to Source

For each vulnerability you exploited in Days 1-6, open the source code and find the exact lines. For crAPI (Python/Java), clone the repo and grep for the vulnerable patterns.

```
git clone https://github.com/OWASP/crAPI.git
grep -rn 'raw_input\|format\|%s' services/ --include='*.py'
grep -rn 'SELECT.*+' services/ --include='*.py'   # raw SQL concat
```

Vulnerability	Code Pattern to Find
SQLi	Look for: string concatenation in DB queries, .format() in SQL strings, raw SQL without parameterization
BOLA	Look for: endpoints that take object IDs but never call check_ownership() or compare to request.user.id
Mass Assignment	Look for: model(**request.json()) or direct dict unpacking into ORM models
SSRF	Look for: requests.get(user_controlled_url) with no URL scheme/host validation

Task 2: Draft Theoretical Patches

SQLi Patch

```
# VULNERABLE
query = "SELECT * FROM users WHERE email='" + email + "'"
# PATCHED (parameterized)
query = "SELECT * FROM users WHERE email=?"
cursor.execute(query, (email,))
```

BOLA Patch

```
# VULNERABLE
def get_order(order_id):
    return Order.query.get(order_id)
# PATCHED
def get_order(order_id, current_user):
    order = Order.query.get_or_404(order_id)
    if order.user_id != current_user.id:
        abort(403)
    return order
```

Mass Assignment Patch

```
# VULNERABLE
user = User(**request.json())
# PATCHED (explicit whitelist)
allowed = ['name', 'email', 'password']
data = {k: v for k, v in request.json().items() if k in allowed}
user = User(**data)
```

WEEK 2 — Target Construction & Threat Modelling

Now you switch from attacker to builder. You are constructing a deliberately vulnerable API that you will spend Weeks 3-8 analysing, patching, and hardening. Every bug you plant must be intentional and traceable.

Task 1: Build the Vulnerable API

Tech Stack

- Python 3.10+
- FastAPI — async REST framework
- SQLite (dev) / PostgreSQL (prod)
- Docker + Docker Compose

Project Structure

```
vuln-api/
  app/
    main.py          # FastAPI app
    models.py        # SQLAlchemy ORM models
    routes/
      auth.py        # SQLi here
      users.py       # BOLA here
      fetch.py       # SSRF here
  Dockerfile
  docker-compose.yml
  requirements.txt
```

Intentional SQLi — auth.py

```
from fastapi import APIRouter, Depends
import sqlite3

router = APIRouter()

@router.post('/login')
def login(username: str, password: str):
    # INTENTIONAL SQLi: raw string concatenation
    conn = sqlite3.connect('app.db')
    query = f"SELECT * FROM users WHERE username='{username}'
              AND password='{password}'"
    result = conn.execute(query).fetchone()
    if result:
        return {'token': 'fake-jwt-' + result[0]}
    return {'error': 'Invalid credentials'}
```

Intentional BOLA — users.py

```
@router.get('/users/{user_id}/profile')
def get_profile(user_id: int, current_user=Depends(get_current_user)):
    # INTENTIONAL BOLA: no ownership check
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(404)
    return user  # returns ANY user's profile, no auth check
```

Intentional SSRF — fetch.py

```
import httpx

@router.get('/fetch')
async def fetch_url(url: str):
    # INTENTIONAL SSRF: no URL validation
    async with httpx.AsyncClient() as client:
        resp = await client.get(url)
    return {'content': resp.text[:500]}
```

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

docker-compose.yml

```
version: '3.9'
services:
  api:
    build: .
    ports:
      - '8000:8000'
    volumes:
      - ./app.db:/app/app.db
```

Run it

```
docker-compose up --build
curl http://localhost:8000/docs   # FastAPI Swagger UI
```

Task 2: Threat Model

What is a Threat Model?

A threat model is a structured document that answers: what are we building, what could go wrong, and what are we doing about it? It forces you to think like an attacker before writing production code.

STRIDE Framework

Threat Category	Description & Mitigation
`Spoofing`	Pretending to be someone else. Mitigated by: strong authentication.
`Tampering`	Modifying data in transit or at rest. Mitigated by: integrity checks, HTTPS.
`Repudiation`	Denying an action was taken. Mitigated by: audit logging.
`Information Disclosure`	Exposing data to unauthorized parties. Mitigated by: authorization checks (BOLA fix), encryption.
`Denial of Service`	Making system unavailable. Mitigated by: rate limiting, circuit breakers.
`Elevation of Privilege`	Gaining higher access than authorized. Mitigated by: mass assignment fix, RBAC.

Data Flow Diagram for your API

Draw and document: Browser -> [HTTPS] -> FastAPI -> [SQL] -> SQLite/Postgres. Mark trust boundaries: external (untrusted) vs internal (trusted). Mark where each vulnerability class sits on the DFD.

WEEK 3 — SAST Integration & Baseline Scanning

Static Application Security Testing (SAST) analyses source code without running it. You are building an automated scanner that will catch vulnerabilities every time someone opens a PR.

Semgrep: Concept & Mechanics

Semgrep is a pattern-matching engine for source code. You write rules in YAML that describe syntactic patterns (like 'string concatenated into SQL query') and Semgrep finds every occurrence across your codebase.

How Semgrep Works

- Parses source code into an AST (Abstract Syntax Tree)
- Matches your YAML pattern against AST nodes
- Reports file, line number, and matched code
- Unlike grep: understands code structure, not just text

Install

```
pip install semgrep
semgrep --version
```

Run Existing Rulesets

```
# Scan with OWASP top 10 ruleset
semgrep --config=p/owasp-top-ten ./app/

# Python security ruleset
semgrep --config=p/python-security ./app/

# Output JSON for later processing
semgrep --config=p/owasp-top-ten --json ./app/ > semgrep_results.json
```

Write a Custom SQLi Detection Rule

```
# rules/sqli.yaml
rules:
  - id: raw-sql-fstring
    patterns:
      - pattern: |
          $CONN.execute(f"...{$VAR}...")
      - pattern: |
          $CONN.execute("..." + $VAR + "...")
    message: |
```

```
      Potential SQL injection: user input concatenated into query.
      Use parameterized queries: conn.execute(query, (var,))
    severity: ERROR
    languages: [python]
```

Write a Custom BOLA Detection Rule

```
# rules/bola.yaml
rules:
  - id: missing-ownership-check
    patterns:
      - pattern: |
          @router.get(".../{$ID}...")
          def $FUNC($ID: int, ...):
              ...
              db.query(...).filter(...$ID...).first()
              ...
      - pattern-not: |
          if $OBJ.$FIELD != $USER.$FIELD:
              ...
    message: Route accesses objects by ID without visible ownership check.
    severity: WARNING
    languages: [python]
```

Understanding False Positives vs True Positives

Category	Definition & Action
`True Positive`	Semgrep flags actual vulnerable code. Verify: trace the data flow manually.
`False Positive`	Semgrep flags safe code (e.g., parameterized query that looks like concat). Add // nosemgrep comment or refine the rule pattern.
`False Negative`	Vulnerable code that Semgrep misses. Improve rule pattern coverage.

GitHub Actions CI Pipeline

```
# .github/workflows/sast.yml
name: SAST Scan
on: [pull_request]
jobs:
  semgrep:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run Semgrep
        run: |
          pip install semgrep
          semgrep --config=p/owasp-top-ten \
                  --config=rules/ \
                  --error ./app/
```

```
# --error: exit code 1 if findings > 0 (fails the build)
```

WEEK 4 — Advanced Static Analysis & Taint Flow (CodeQL)

CodeQL is GitHub's semantic code analysis engine. Unlike Semgrep's pattern matching, CodeQL builds a full database of your code's AST and data flow graph, then lets you query it with a SQL-like language to trace tainted data from source to sink.

Taint Analysis: The Core Concept

Taint analysis tracks data from a taint source (user-controlled input) to a taint sink (dangerous function call). If there is a path from source to sink with no sanitization, that is a vulnerability.

Concept	Description
`Source`	Where untrusted data enters. Example: request.query_params, request.body, argv
`Sink`	Where dangerous operations happen. Example: cursor.execute(), subprocess.run(), requests.get()
`Sanitizer`	Function that makes tainted data safe. Example: parameterized query, html.escape(), URL whitelist check
`Flow`	A path from source to sink through function calls. CodeQL traces these across files.

Install CodeQL CLI

```
# Download from: https://github.com/github/codeql-action/releases
# Extract to ~/codeql
export PATH=$PATH:~/codeql
codeql version
```

Build a CodeQL Database

```
# For Python (interpreted — use trace mode)
codeql database create vuln-api-db \
    --language=python \
    --source-root=./app/
```

Run Built-in Security Queries

```
codeql database analyze vuln-api-db \
    --format=sarif-latest \
    --output=results.sarif \
    codeql/python-queries:Security/

# View results
```

```
cat results.sarif | python3 -m json.tool | grep -A5 'ruleId'
```

Write a Custom CodeQL Query for BOLA

```
// bola_detection.ql
import python
import semmle.python.dataflow.new.DataFlow
import semmle.python.ApiGraphs

// Find route handlers that accept user-controlled IDs
// but do not call an ownership check
from Function handler, Parameter id_param
where
  handler.getName().matches("%get%") and
  id_param = handler.getArgByName("user_id") and
  not exists(IfStmt ownerCheck |
    ownerCheck.getEnclosingFunction() = handler
  )
select handler, "Route handler accesses user_id without ownership check"
```

Correlating Findings

For each CodeQL finding, open the SARIF output and note: file, startLine, endLine, ruleId. Cross-reference with Semgrep output from Week 3. A finding that appears in both tools with high severity is a confirmed true positive that needs patching.

WEEK 5 — Dynamic Verification: Fuzzing with Atheris

Static analysis finds potential vulnerabilities in code paths. Fuzzing actually exercises the running code with malformed inputs and detects crashes, exceptions, and unexpected behaviour at runtime. It verifies that static findings are truly exploitable.

Atheris: Coverage-Guided Python Fuzzer

How Fuzzing Works

- Atheris generates mutated byte sequences as inputs
- It instruments your code to track which branches are executed (coverage)
- When a new input exercises a new code branch, it's saved as 'interesting'
- The fuzzer learns which mutations explore more code, amplifying coverage over time
- Crashes, assertion errors, and uncaught exceptions are saved as corpus items

Install

```
pip install atheris
# Requires: clang, libfuzzer
sudo apt install clang
```

Basic Atheris Harness for SQL Parser

```
import atheris
import sys

# Import the code under test
sys.path.insert(0, './app')
from routes.auth import login_logic

@atheris.instrument_func
def TestOneInput(data):
    # Convert raw bytes to string
    fdp = atheris.FuzzedDataProvider(data)
    username = fdp.ConsumeUnicodeNoSurrogates(20)
    password = fdp.ConsumeUnicodeNoSurrogates(20)
    try:
        login_logic(username, password)
    except ValueError:
        pass  # Expected error
    # Any OTHER exception = potential bug

atheris.Setup(sys.argv, TestOneInput)
atheris.Fuzz()
```

Run the Fuzzer

```
python3 fuzz_login.py -max_total_time=60
# -max_total_time: stop after 60 seconds
# Crashes saved to ./crash-*
```

Atheris Harness for SSRF Fetch

```
@atheris.instrument_func
def TestOneInput(data):
    fdp = atheris.FuzzedDataProvider(data)
    url = fdp.ConsumeUnicodeNoSurrogates(100)
    try:
        # Test that URL validation doesn't crash or allow internal access
        result = fetch_url_logic(url)
        # If it returns content from 127.0.0.1: SSRF confirmed
        if '127.0.0.1' in result or 'localhost' in result:
            raise AssertionError(f'SSRF: fetched internal URL: {url}')
    except (ValueError, httpx.RequestError):
        pass
```

Correlate Crashes with Static Findings

For every crash corpus item from Atheris, open the file, decode the input, and reproduce manually. Then find the corresponding CodeQL/Semgrep finding. Document: input that triggered the crash, code path executed, root cause vulnerability, SAST rule that flagged it.

Dynamic + Static = Confidence

Static analysis says 'this pattern looks dangerous'. Dynamic fuzzing says 'this pattern is actually reachable and crashing with this input'. Together they give you a verified, exploitable vulnerability with an exact test case.

WEEK 6 — Formal Engineering Patches

You have found, verified, and documented every vulnerability class. Now you remove them systematically — not ad-hoc fixes, but structural refactors that make the vulnerability class impossible by design.

Patch 1: Eliminate SQLi — ORM Migration

Why ORM Eliminates SQLi

An ORM (Object Relational Mapper) like SQLAlchemy never builds SQL strings from user input. It uses bound parameters internally — the database driver handles escaping at the protocol level. You lose the ability to create SQLi by construction.

Install SQLAlchemy

```
pip install sqlalchemy[asyncio] aiosqlite
```

Replace Raw SQL with ORM

```
# BEFORE (vulnerable)
def login(username, password):
    conn = sqlite3.connect('app.db')
    q = f"SELECT * FROM users WHERE username='{username}'"
    return conn.execute(q).fetchone()

# AFTER (patched — SQLAlchemy ORM)
from sqlalchemy.orm import Session

def login(username: str, password: str, db: Session):
    user = db.query(User)\
             .filter(User.username == username)\
             .filter(User.password == hash_pw(password))\
             .first()
    return user
```

If ORM is not possible: Parameterized Queries

cursor.execute('SELECT * FROM users WHERE username=?', (username,)) — the ? placeholder is the minimum safe baseline. The driver escapes the value before sending to the DB.

Patch 2: Eliminate BOLA — Centralized Auth Middleware

Pattern: Ownership Middleware

Instead of checking ownership in every individual route handler (which developers forget), write a dependency that is injected into every relevant route. It runs before the handler and raises 403 if ownership fails.

```
# middleware/auth.py
from fastapi import Depends, HTTPException
from .models import User, Order

def require_owner(resource_type: str):
    def dependency(resource_id: int,
                   current_user: User = Depends(get_current_user),
                   db: Session = Depends(get_db)):
        if resource_type == 'order':
            obj = db.query(Order).get_or_404(resource_id)
            if obj.user_id != current_user.id:
                raise HTTPException(status_code=403,
                                    detail='Forbidden')
            return obj
    return dependency

# routes/orders.py — patched route
@router.get('/orders/{order_id}')
def get_order(order = Depends(require_owner('order'))):
    return order  # ownership already verified by dependency
```

Patch 3: Eliminate Mass Assignment — Pydantic Schemas

```
# schemas.py — explicit input whitelist
from pydantic import BaseModel

class UserCreate(BaseModel):
    name: str
    email: str
    password: str
    # isAdmin, credit, role are NOT here — they cannot be submitted

# route — only accepts whitelisted fields
@router.post('/register')
def register(user_in: UserCreate, db: Session = Depends(get_db)):
    user = User(**user_in.dict())  # safe: only whitelisted fields
    db.add(user)
```

Patch 4: Eliminate SSRF — URL Allowlist

```
from urllib.parse import urlparse

ALLOWED_SCHEMES = {'https'}
ALLOWED_HOSTS = {'api.partner.com', 'cdn.example.com'}
```

```
def validate_url(url: str) -> str:
    parsed = urlparse(url)
    if parsed.scheme not in ALLOWED_SCHEMES:
        raise ValueError(f'Scheme {parsed.scheme} not allowed')
    if parsed.netloc not in ALLOWED_HOSTS:
        raise ValueError(f'Host {parsed.netloc} not allowed')
    return url
```

WEEKS 7-8 — Pipeline Enforcement & Final Evaluation

Week 7: CI/CD Enforcement Gates

Philosophy: Shift Left

Security gates in CI/CD mean vulnerabilities are caught before they ever reach production. A PR that introduces a high-severity SAST finding fails the build — it cannot be merged. This is security enforcement at the process level.

Full GitHub Actions Pipeline

```
# .github/workflows/security-pipeline.yml
name: Security Pipeline
on: [push, pull_request]

jobs:
  semgrep:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: pip install semgrep
      - run: semgrep --config=p/owasp-top-ten --error ./app/

  codeql:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: github/codeql-action/init@v2
        with:
          languages: python
      - uses: github/codeql-action/analyze@v2
        with:
          category: /language:python

  fuzz:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: pip install atheris
      - run: timeout 120 python3 fuzz/fuzz_login.py || true
      - run: |
          if ls crash-* 1>/dev/null 2>&1; then
            echo 'FUZZER CRASH FOUND'
            exit 1
          fi
```

```
  auth-unit-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: pip install pytest
      - run: pytest tests/test_auth_middleware.py -v
```

Authorization Middleware Unit Tests

```
# tests/test_auth_middleware.py
import pytest
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_bola_rejected():
    # User A token
    token_a = login_and_get_token('userA', 'pw')
    # Try to access User B's resource
    resp = client.get('/orders/999',
                      headers={'Authorization': f'Bearer {token_a}'})
    assert resp.status_code == 403

def test_sqli_payload_rejected():
    resp = client.post('/login',
                       json={'username': "' OR '1'='1",
                             'password': 'x'})
    assert resp.status_code != 200
    assert 'token' not in resp.json()
```

Week 8: Final Security Evaluation Report

Report Structure

Section	Contents
1. Executive Summary	1-page overview: scope, critical findings count, risk level, key recommendations
2. Threat Model	Architecture diagram, DFD, trust boundaries, STRIDE analysis table
3. Vulnerability Findings	For each bug: description, CVSS score, code location, proof-of-concept, SAST tool that found it
4. Root Cause Analysis	Why each vulnerability existed: design flaw, missing control, insecure default
5. Patches Applied	Before/after code diff, explanation of structural fix, why it prevents the class

6. CI/CD Pipeline	Gate configuration, test coverage, metrics (false positive rate, scan time)
7. Architectural Improvements	Recommendations beyond the immediate fixes: RBAC design, secrets management, API rate limiting

CVSS Scoring (use for each finding)

Score Range	Typical Findings & Priority
Critical 9.0-10.0	Unauthenticated RCE, full data exfiltration. Patch immediately.
High 7.0-8.9	SQLi with data access, BOLA leaking PII. Patch before next release.
Medium 4.0-6.9	SSRF to internal metadata, mass assignment. Patch within sprint.
Low 0.1-3.9	Information disclosure, verbose errors. Schedule for backlog.

Skills Inventory After Week 8

By completing this pipeline you will demonstrate competency in:

Skill	Specific Competency
Manual Web Exploitation	SQLi, XSS, BOLA, SSRF with Burp Suite
Binary Exploitation	Stack buffer overflow, EIP/RIP control, shellcode redirection
API Security	Black-box surface mapping, BOLA chains, mass assignment
Threat Modelling	DFD, STRIDE, attack surface documentation
SAST — Semgrep	Rule writing, TP/FP triage, custom pattern detection
SAST — CodeQL	DB creation, taint flow queries, custom QL rules
Dynamic Fuzzing	Atheris harness engineering, corpus analysis, crash triage
Secure Engineering	ORM migration, ownership middleware, Pydantic whitelisting, SSRF allowlist
DevSecOps	GitHub Actions multi-stage pipeline, build gates, unit test enforcement
AppSec Reporting	CVSS scoring, RCA, structured professional security report

QUICK REFERENCE — Commands & Cheatsheets

Essential Commands by Week

Week	Key Commands
Week 1	Burp proxy on 127.0.0.1:8080 | Narnia SSH: ssh -p 2226 narnia0@narnia.labs.overthewire.org | crAPI: docker-compose up
Week 2	docker-compose up --build | uvicorn app.main:app --reload | curl http://localhost:8000/docs
Week 3	semgrep --config=p/owasp-top-ten ./app/ | semgrep --config=rules/ --json ./app/ > out.json
Week 4	codeql database create db --language=python | codeql database analyze db --format=sarif-latest
Week 5	pip install atheris | python3 fuzz_target.py -max_total_time=120 | ls crash-*
Week 6	pip install sqlalchemy[asyncio] | pytest tests/ -v | git diff HEAD~1 -- app/
Week 7	git push origin feature-branch (triggers CI) | gh run list | gh run view <id>
Week 8	Compile SARIF outputs + manual findings into report | cvss-calculator for scoring

Burp Suite Key Shortcuts

Shortcut	Action
Ctrl+R	Send request to Repeater
Ctrl+I	Send request to Intruder
Ctrl+Shift+P	Open Proxy Intercept toggle
Ctrl+F	Search in current tab
Right-click > Copy as curl	Copy request as curl command for scripting